Frontiers in Computing: Discussion 7

Rento Saijo

2026-10-14

From a question to an answer

- Leo Cui begins with a simple dinner question typed into a phone. Its quick answer makes a data center seem almost invisible, yet the exchange depends on a network, a trained model, and machines that can run that model on demand. Training adjusts a model's parameters using large amounts of data and computation; answering this particular question is *inference*, when those learned parameters are used to predict a response. Cui contrasts a search engine retrieving indexed pages with a language model generating text. The contrast helps explain why generation requires fresh computation, although a real AI application can also search documents or reuse cached work. The model assembles its answer for the current context through new computation.
- After the phone sends its request over wireless and fiber links, an entry service may authenticate the user, limit abusive traffic, and assemble the instructions and conversation context supplied to the model. A tokenizer then converts the text into numbered tokens: pieces of words, whole words, punctuation, or other text fragments, depending on the tokenizer. The accelerator receives numerical representations of those tokens alongside any supplied instructions and previous messages. Requests can travel through several network and software services before reaching a model worker. Each step contributes to the delay a user experiences, so the apparent speed of the answer reflects more than the raw speed of a GPU.
- Model execution has two useful phases. During *prefill*, the model processes many prompt tokens in parallel and calculates attention information for the existing context. A key–value (KV) cache retains prior attention keys and values so later steps do not have to recompute them from the beginning. It stores machine-readable attention states, and a serving system may reuse them for a compatible prompt prefix. During *decode*, the model predicts a new token from the prompt and tokens already generated, adds it to the context, and repeats. This sequential dependence helps explain why a long response can take time even after its first token appears. A serving application can stream decoded text back to the phone as it becomes available; the display need not update at exactly one word per step, since tokens and words do not coincide.
- The worker may also share its accelerator with other requests. Batching improves utilization by scheduling compatible work together, while reuse of an already computed prompt prefix can avoid some prefill work. Quantization stores or computes some values at lower precision to reduce memory and processing demands, provided the resulting quality is acceptable. These methods have workload-dependent tradeoffs: larger batches can improve throughput while increasing a request's wait, and a cache helps only when content can actually be reused. For a question about a company's private files, retrieval-augmented generation can add relevant documents to the prompt before generation. In that case, searching and writing cooperate; the answer still depends on the retrieved material, model behavior, and access rules.

The machines beneath the model

- The model worker runs on a server whose components do different jobs. CPUs execute the operating system and coordinate application logic, input and output, and accelerator work. GPUs perform many suitable numerical operations in parallel, making them useful for the matrix calculations common in neural networks; CPUs remain essential even when GPUs dominate the arithmetic. A server also has system memory, network interfaces, storage, and power supplies. Cui's factory analogy makes the division of labor memorable, but there is no single required rack design or fixed number of GPUs for an AI request. A smaller model may fit on one accelerator, while a larger model or a demanding training job may spread work across many devices. What matters is whether the components can keep one another supplied with data.
- Memory illustrates that dependence. GPU high-bandwidth memory holds model weights and often active KV caches close to the computation; its capacity and rate of transfer can limit how many requests run together and how quickly tokens are produced. Server DRAM provides a different, larger pool for the CPU and supporting processes. Solid-state drives persist model files and frequently used data, while hard drives may economically hold less frequently accessed bulk data such as training sets or checkpoints. These layers differ in speed, capacity, and cost, and a request can stall when the needed information has to move from a slower layer. Whether conversations or generated output are retained depends on application policy. The memory hierarchy therefore helps explain both the speed of a response and the cost of providing one.
- Networking extends that hierarchy beyond a single server. Fast links may join accelerators within a machine or rack; switches and fiber connect racks and storage across a facility. Bandwidth measures how much data a link carries over time, whereas latency measures the delay before a transfer arrives. Either can matter when a model is divided among devices, since intermediate results have to reach the next computation. Ethernet and specialized fabrics such as InfiniBand offer different engineering choices, and optical transceivers convert signals for fiber paths where appropriate. The same network also carries user requests, retrieved context, model files, and responses, though operators may separate these traffic classes. Adding more GPUs helps little if communication leaves them waiting. A smaller model can answer on one accelerator; the architecture follows the model and workload.
- Power and cooling complete the physical path. A facility needs grid capacity, transformers and distribution equipment, and often backup power that can bridge or survive a supply interruption. Computing equipment converts most of the electricity it uses into heat, so air handling, rear-door heat exchangers, direct-to-chip liquid loops, or other methods must remove that heat at the rack's actual density. A coolant distribution unit can transfer heat from chips to a building loop; what happens afterward determines the total electricity and water use. Power usage effectiveness compares a facility's total power with the power delivered to its IT equipment, but it says nothing by itself about emissions, water use, or service reliability. Cui's tour underscores that power availability and cooling can constrain a project before its chips are installed. Redundancy and efficient equipment help only when the entire system is designed and maintained to use them.
- Supplying that electricity is a planning problem as well as an equipment problem. Cui notes that chip deliveries can outrun utility connection studies and transformer procurement. A developer may weigh grid service against nearby generation or long-term power contracts, but on-site generation still needs physical infrastructure and can affect nearby communities. Natural gas can be dispatched when needed but produces emissions; nuclear generation offers steady low-carbon power where available, while variable renewable sources need other supply or storage when output falls. Similarly, a closed liquid loop at the chip can ultimately reject heat through an evaporative tower. The facility and its location therefore set the operating envelope for an AI rack.

Sharing physical infrastructure

- IBM's account places virtualization between those physical machines and the services customers see. The idea reaches back to time-sharing on mainframes: IBM's CP-40 research and later VM/370 showed how one computer could present several separate environments instead of assigning a whole machine to every task. The physical computer is the *host*; a virtual machine (VM) is a *guest* with its own operating system and virtual hardware. A hypervisor allocates host CPU time, memory, storage, and network access among guests and enforces boundaries between them. A Type 1 hypervisor runs directly on the hardware, as is common in server environments. A Type 2 hypervisor runs above an existing host operating system, often to run another OS on a personal computer. Both create useful VMs, but their placement and overhead differ. The virtualization layer is software, while the capacity it divides is still physical.
- Sharing a server can raise utilization when separate applications would otherwise leave their own machines idle. Administrators can define VMs from templates, provision them quickly, change their allocations, and retire them when work ends. This flexibility makes it easier to create a development environment or expand a cloud service, and fewer underused servers can reduce equipment and operating costs. Those gains are conditional: guests can compete for finite resources, a large VM can still waste capacity, and management software itself takes work to run. IBM also describes moving, replicating, or restoring VMs for maintenance and disaster recovery. A snapshot can help roll back a system, yet it is not a substitute for a protected backup; failover requires another healthy host, accessible data, and a tested procedure. Virtualization supplies options for resilience rather than an automatic guarantee of uptime.
- Isolation also requires continuing administration. Different guest operating systems can share one server without sharing the same memory space, which helps contain an application failure and gives operators distinct policy boundaries. Yet the hypervisor, management accounts, and underlying hardware remain common dependencies. IBM therefore pairs VM security with access controls, updates, network segmentation, encryption, and monitoring. A team might keep a test VM away from production data, limit who can change its virtual network, and restore a clean copy after an incident. The safety of a virtual environment depends on those measures and their enforcement. They also matter when a provider hosts several customers on the same physical platform.
- The same abstraction extends beyond whole servers. CPU virtualization presents virtual processors to guests, while GPU passthrough can give a guest one physical accelerator and shared virtual GPUs can divide suitable accelerator capacity among guests. These choices affect isolation, scheduling, and performance, so a large AI job may receive dedicated hardware while lighter work shares it. Storage virtualization pools physical devices into logical capacity that can be assigned to machines; network virtualization gives administrators software-defined connections and functions such as virtual switches or firewalls. Together, these layers can present separate virtual data-center environments over shared racks. Their logical boundaries still rest on disks, cables, processors, and power. We can see the connection to Cui's request journey: the model worker may appear as an isolated service, even though its compute and network paths ultimately occupy equipment beside other workloads.
- IBM also describes forms of virtualization farther from the AI rack. A virtual desktop can run centrally and send its interface to a user's device, or a local hypervisor can run a second desktop OS on a laptop. Application virtualization delivers software without installing it in the usual way on the user's operating system; data virtualization presents information from multiple sources through a common access layer. These solve different problems from splitting a GPU, but each separates the interface a user sees from the physical or software arrangement underneath. Cloud services build on several such abstractions: infrastructure as a service exposes configurable virtual compute, storage, and networking; platform as a service adds managed development components; software as a service delivers the finished application. Each service arrangement assigns different responsibilities to the customer and provider; the hypervisor remains one of several enabling technologies.

Containers, control, and capacity

- IBM's embedded containerization explainer uses a small Node.js application to show why teams sometimes choose a lighter unit than a VM. Each VM includes a guest operating system as well as the application and its dependencies. Containers package an application and needed libraries while sharing a host OS kernel through a container runtime, so several instances can avoid duplicating a whole guest OS. A developer describes the environment in a build file, creates an image, and runs that image as a container. The image helps carry the same dependencies from development into deployment, reducing one common source of incompatibility. Portability still depends on a compatible host kernel and processor architecture, correct configuration, and access to external services. VMs offer a stronger OS boundary and can run different guest operating systems; containers offer a compact way to deploy compatible applications.
- In the explainer's next step, a Python service calls an image-recognition API for the Node.js application. If these parts are deployed separately, operators can add Node.js instances when front-end traffic rises without automatically adding identical numbers of Python instances. Containers make that modular pattern convenient, but separate VMs could also scale independently when designed that way. The useful lesson is to match each component's deployment unit to its demand. At data-center scale, orchestration software can place containers on available hosts, restart failed instances, and distribute requests. Those hosts may themselves be VMs, and the containers' processes still use real CPU time, memory, network bandwidth, and sometimes GPUs. The layers complement each other: virtualization organizes and isolates capacity, while containers package and operate the applications that consume it.
- Software makes the physical and virtual layers act as one service. Linux runs many of the servers, GPU programming tools let applications use accelerators, and model-serving software schedules prefill and decode work, manages caches, and returns output to the network. Container orchestration can add or remove workers as demand changes within the facility's available electricity and cooling capacity. Security similarly spans layers: the gateway needs authentication and rate limits, hypervisors and runtimes need updates and isolation, virtual networks need access controls, and retrieved company documents require permissions of their own. A VM snapshot is useful for recovery, while encryption and backups protect different risks. We should judge an AI service by this combined path from user to chip and back, since a failure or unsafe configuration at any layer can undermine an otherwise capable model.
- An illustrative deployment brings these choices together. CPU-based gateway and document-retrieval services could run in separate containers, while model workers use GPU-equipped hosts with access to the accelerator arrangement their workload needs. Virtual networks would restrict which services can read private documents; storage would hold model files and approved application data; orchestration would add workers as traffic changes. A provider might place the containers inside VMs for another isolation boundary, or reserve a GPU for a large job instead of sharing it. The point is to allocate each resource deliberately. A busy gateway cannot compensate for too few model workers, and idle GPUs do not help when retrieval, memory transfer, or the network is the limiting step.
- Cui closes by asking whether demand for useful AI services will justify the cost of building and operating their infrastructure. That is an investor's question rather than a settled forecast. Serving improvements may reduce the computation or memory needed per answer, yet more users, longer prompts, and richer applications can increase total demand. Hardware also ages, while power connections, cooling systems, and buildings have their own lead times and costs. The virtualization material adds a practical response: pool resources when sharing is efficient, isolate workloads when needed, and choose VMs, containers, or dedicated accelerators according to the job. Those choices still leave the facility responsible for electricity, water, and maintenance. The quick dinner suggestion on a phone is therefore the visible end of a larger design problem: turning limited physical capacity into a reliable, secure, and economically worthwhile service.